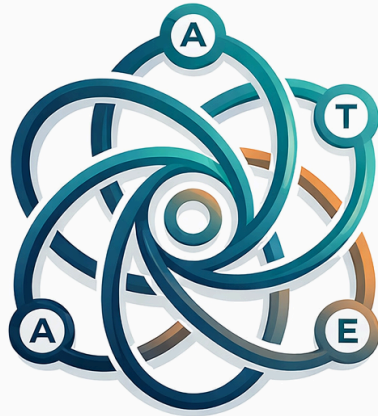




NEXUS

SISTEMA DE GESTIÓN DE PRÁCTICAS ACADÉMICAS

Iker Acevedo Donate

CampusFP

Memoria TFG



ÍNDICE

1. De qué trata mi idea y por qué la propongo.....	4
2. Objetivos del proyecto.....	5
3. Análisis de Requisitos (Especificaciones Técnicas).....	5
3.1 Requisitos Funcionales (RF).....	5
3.2 Requisitos No Funcionales (RNF).....	6
4. Arquitectura y Tecnologías.....	7
4.1. Diseño de la Lógica (Backend).....	9
4.2. El módulo de seguimientos: cuando el diseño inicial no reflejaba la realidad.....	12
4.3. Seguridad aplicada: revisión sistemática OWASP.....	13
4.4. Tests automatizados y cobertura de código.....	14
5. Diseño de la Interfaz por Roles.....	16
5.1. Panel del Alumno.....	16
5.2. Panel del Tutor.....	17
5.3. Panel del Centro Educativo / Admin.....	18
5.4. Panel del Tutor de Empresa.....	19
5.5. Los cuatro paneles en versión móvil.....	21
6. Modelo de la Base de Datos.....	21
7. Planificación (Cómo lo voy a hacer).....	23
8. Conclusión y Futuro.....	25



1. De qué trata mi idea y por qué la propongo

La idea de este proyecto surge de un problema que casi todos los estudiantes y tutores hemos vivido: el caos administrativo que suponen las prácticas. Actualmente, la gestión está muy fragmentada. El alumno está en la empresa, el tutor en el centro y la comunicación entre ellos suele ser un lío de correos electrónicos que se pierden, llamadas o documentos de Excel que van y vienen. Esto hace que sea muy difícil llevar un control real de lo que el alumno hace día a día.

Mi aplicación busca centralizar todo esto en un único entorno digital. Aunque la plataforma está pensada para gestionar todo el ciclo (desde que una empresa publica una oferta hasta que se firma el convenio), **este TFG se va a centrar sobre todo en la parte del seguimiento y la comunicación**. Me parece la parte más importante porque es donde más fallos hay actualmente. Quiero que el tutor sepa de verdad cómo le va al alumno y que el alumno tenga un sitio fácil donde reportar cualquier problema sin que se quede en el olvido.

2. Objetivos del proyecto

- **Comunicación centralizada:** Crear un chat interno vinculado a la práctica para que las dudas y avisos no se mezclen con el correo personal.
- **Seguimiento real:** Que el alumno registre sus tareas y sus horas semanalmente de forma sencilla.
- **Gestión de incidencias:** Si pasa algo en la empresa, el alumno puede dar la alarma con un botón y el tutor recibe un aviso directo.
- **Supervisión del centro:** Que el centro educativo tenga una pantalla donde ver el estado de todos sus alumnos a la vez.



3. Análisis de Requisitos (Especificaciones Técnicas)

Para complementar los objetivos anteriores, he definido los requisitos técnicos que debe cumplir el sistema en esta primera fase:

3.1 Requisitos Funcionales (RF)

- **RF-01 (Gestión de Usuarios):** El sistema debe permitir el registro y login de usuarios con diferentes roles (Alumno, Tutor, Admin).
- **RF-02 (Perfil de Usuario):** El usuario debe poder visualizar sus datos de perfil una vez autenticado mediante token.
- **RF-03 (Gestión de Maestros):** Capacidad de listar Centros Educativos y Empresas colaboradoras para poblar los formularios del sistema.
- **RF-04 (Gestión de Prácticas):** El sistema permite la creación, consulta y actualización de convenios de prácticas por parte de administradores y tutores.
- **RF-05 (Seguridad por Roles):** Implementación de niveles de acceso estrictos; los alumnos solo acceden a su información y los administradores gestionan el sistema completo.
- **RF-06 (Seguimiento Diario):** El alumno puede registrar cada día qué ha hecho, cuántas horas ha trabajado y una descripción de las tareas. La aplicación lleva la cuenta automáticamente de las horas acumuladas frente al total del convenio.
- **RF-07 (Validación en Dos Pasos):** Los partes del alumno tienen que pasar primero por el tutor de empresa y después por el tutor del centro, igual que ocurre con los papeles actualmente. Si la empresa rechaza un parte, el sistema avisa automáticamente al tutor del centro.
- **RF-08 (Gestión de Ausencias):** El alumno puede registrar ausencias con fecha y motivo, y adjuntar un fichero justificante (PDF, JPG o PNG de hasta 5 MB). El tutor de empresa revisa cada ausencia y la clasifica como justificada o injustificada. El sistema valida que no exista ya una ausencia para la misma práctica y fecha, evitando duplicados.
- **RF-09 (Chat en tiempo real):** Cada práctica dispone de un canal de chat privado entre el alumno y sus tutores. Los mensajes nuevos se reciben en tiempo real mediante WebSocket sin necesidad de recargar la pantalla. Al abrir el chat se carga el historial completo a través de REST y los mensajes posteriores llegan por suscripción al canal correspondiente.
- **RF-10 (Sistema de auditoría):** El sistema registra automáticamente todas las operaciones relevantes —creación, modificación y validación de prácticas, seguimientos, ausencias e incidencias, así como inicios y cierres de sesión— en una tabla centralizada. El administrador puede consultarlas filtrando por módulo, con el email del actor y una descripción de la acción ejecutada.
- **RF-11 (Gestión de incidencias):** El alumno puede reportar cualquier problema durante la práctica mediante un formulario categorizado. Si el tutor de empresa rechaza un parte



de seguimiento, el sistema crea automáticamente una incidencia visible para el tutor del centro, sin que el alumno tenga que hacer nada adicional.

- **RF-12 (Restauración de sesión):** Cuando el usuario reabre la aplicación, esta comprueba si el token JWT almacenado de forma segura sigue siendo válido mediante una llamada a `GET /auth/me`. Si el token es válido, el usuario accede directamente a su panel sin necesidad de volver a introducir sus credenciales. Si el token ha expirado o ha sido revocado, el almacenamiento se limpia y se muestra la pantalla de login.

3.2 Requisitos No Funcionales (RNF)

- **RNF-01 (Seguridad):** Cifrado BCrypt para contraseñas, autenticación stateless mediante JWT con claim `jti` único por token, blacklist en servidor para invalidación inmediata en logout y limpieza automática de entradas expiradas. Rate limiting de diez peticiones por IP y minuto en los endpoints de autenticación.
- **RNF-02 (Integridad referencial):** PostgreSQL garantiza la integridad de las relaciones entre entidades. Flyway gestiona el historial de cambios del esquema mediante migraciones versionadas y secuenciales, lo que hace el estado de la base de datos reproducible en cualquier entorno.
- **RNF-03 (Usabilidad y adaptabilidad):** La interfaz se adapta automáticamente al tamaño de pantalla usando `LayoutBuilder`: en dispositivos anchos (más de 600 px) se muestra un sidebar de navegación lateral; en móvil se sustituye por una barra de navegación inferior. La misma base de código Flutter genera ambas variantes sin duplicar lógica.
- **RNF-04 (Portabilidad):** El frontend se distribuye como aplicación web estática compilada. Un Dockerfile multi-stage construye la app con el SDK de Flutter y transfiere únicamente los ficheros estáticos resultantes a una imagen Nginx Alpine, eliminando todas las herramientas de compilación del artefacto final. Esto reduce el tamaño de imagen y la superficie de ataque en producción.
- **RNF-05 (Mantenibilidad):** La arquitectura en capas estricta —Controller, Service, Repository— separa las responsabilidades y facilita la extensión del sistema. Cada módulo funcional tiene su propio servicio, su propio controlador y sus propios tests. Añadir un nuevo módulo no requiere modificar los existentes.
- **RNF-06 (Trazabilidad):** El sistema de auditoría registra cada operación en una transacción independiente mediante `Propagation.REQUIRES_NEW`, de forma que incluso si la operación principal falla y su transacción hace rollback, el intento queda almacenado. Esto garantiza un historial completo e inalterable de lo que ha ocurrido en el sistema.
- **RNF-07 (Confidencialidad del token en cliente):** El token JWT se almacena en el dispositivo usando `flutter_secure_storage`, que utiliza el almacenamiento cifrado del sistema operativo (Keychain en iOS, EncryptedSharedPreferences en Android, credenciales del sistema en web). Nunca se guarda en memoria no cifrada ni en localStorage.
- **RNF-08 (Disponibilidad del entorno):** El orquestador Docker Compose configura un healthcheck sobre la base de datos PostgreSQL antes de permitir que el backend



arranque, evitando errores de conexión durante el inicio en frío. Los tres servicios —base de datos, API y frontend— se despliegan y se detienen de forma coordinada con un único comando.

4. Arquitectura y Tecnologías

Para el desarrollo de este proyecto, he decidido evolucionar la idea inicial hacia una Arquitectura Cliente-Servidor (basada en API). Esta estructura separa completamente los datos del servidor de lo que el usuario ve en la pantalla.

- **Frontend (Aplicación Cliente):** Para la implementación del frontend se ha seguido una arquitectura de capas (Data, Domain, Presentation) sobre Flutter. Se utiliza Dio como cliente HTTP avanzado para la gestión de peticiones REST y Provider como motor de inyección de dependencias y gestión de estado. La seguridad se garantiza mediante interceptores que inyectan automáticamente el token JWT en las cabeceras de cada petición.
- **Arquitectura del frontend en detalle**

La arquitectura del frontend sigue el patrón de tres capas propio de Flutter. La capa de datos contiene los modelos —clases Dart que mapean directamente los JSON de la API— y los servicios, que encapsulan cada llamada HTTP usando Dio. Un interceptor global inyecta el token JWT en la cabecera `Authorization` de cada petición y detecta respuestas 401 para redirigir al login sin intervención del desarrollador.

La capa de presentación se organiza en providers —uno por módulo funcional— que extienden `ChangeNotifier`. Cada provider carga sus datos, gestiona el estado de carga y error, y notifica a los widgets cuando algo cambia. Esta separación hace que los widgets sean puramente visuales: reciben datos del provider y delegan las acciones de vuelta a él, sin lógica de negocio incrustada.

La navegación se implementa con `go_router`, que permite definir rutas con guards: si el usuario no está autenticado y navega a una ruta protegida, el router lo redirige automáticamente al login. Al arrancar la aplicación, antes de mostrar ninguna pantalla, se ejecuta `AuthProvider.init()`, que llama a `GET /auth/me` con el token almacenado. Si el servidor confirma que el token sigue siendo válido, el usuario accede directamente a su panel; si devuelve un 401, el almacenamiento se limpia y se muestra el login. Esto elimina la necesidad de que el usuario se autentique en cada sesión mientras su token esté vigente.



- **Backend (API REST):** He optado por **Spring Boot con Java 21**. He implementado una arquitectura de seguridad Stateless basada en **JWT (JSON Web Tokens)**, eliminando las sesiones en servidor para que el sistema sea más rápido y seguro.
- **Base de Datos:** He utilizado **PostgreSQL** (en lugar de MySQL) por su robustez en el manejo de relaciones complejas. Uso la herramienta **Flyway** para gestionar el historial de cambios de las tablas automáticamente.
- **Infraestructura y despliegue con Docker:** Todo el sistema se despliega mediante Docker Compose, que orquesta tres servicios en una red privada: la base de datos PostgreSQL, la API Spring Boot y el frontend Flutter servido por Nginx.

El Dockerfile del backend es un build multi-stage: la primera etapa usa la imagen oficial de Maven con Java 21 para compilar el proyecto; la segunda etapa copia únicamente el JAR resultante sobre una imagen Alpine mínima. El artefacto final no contiene el código fuente ni las herramientas de compilación.

El Dockerfile del frontend sigue el mismo patrón pero con una particularidad relevante: la URL del backend se inyecta en tiempo de compilación mediante el parámetro `--dart-define=API_URL`. Esto evita que la URL quede grabada en el código fuente y permite configurarla por entorno sin recompilar. El valor se lee en Dart con `String.fromEnvironment('API_URL')` y Docker Compose lo pasa como argumento de build desde el fichero `.env`.

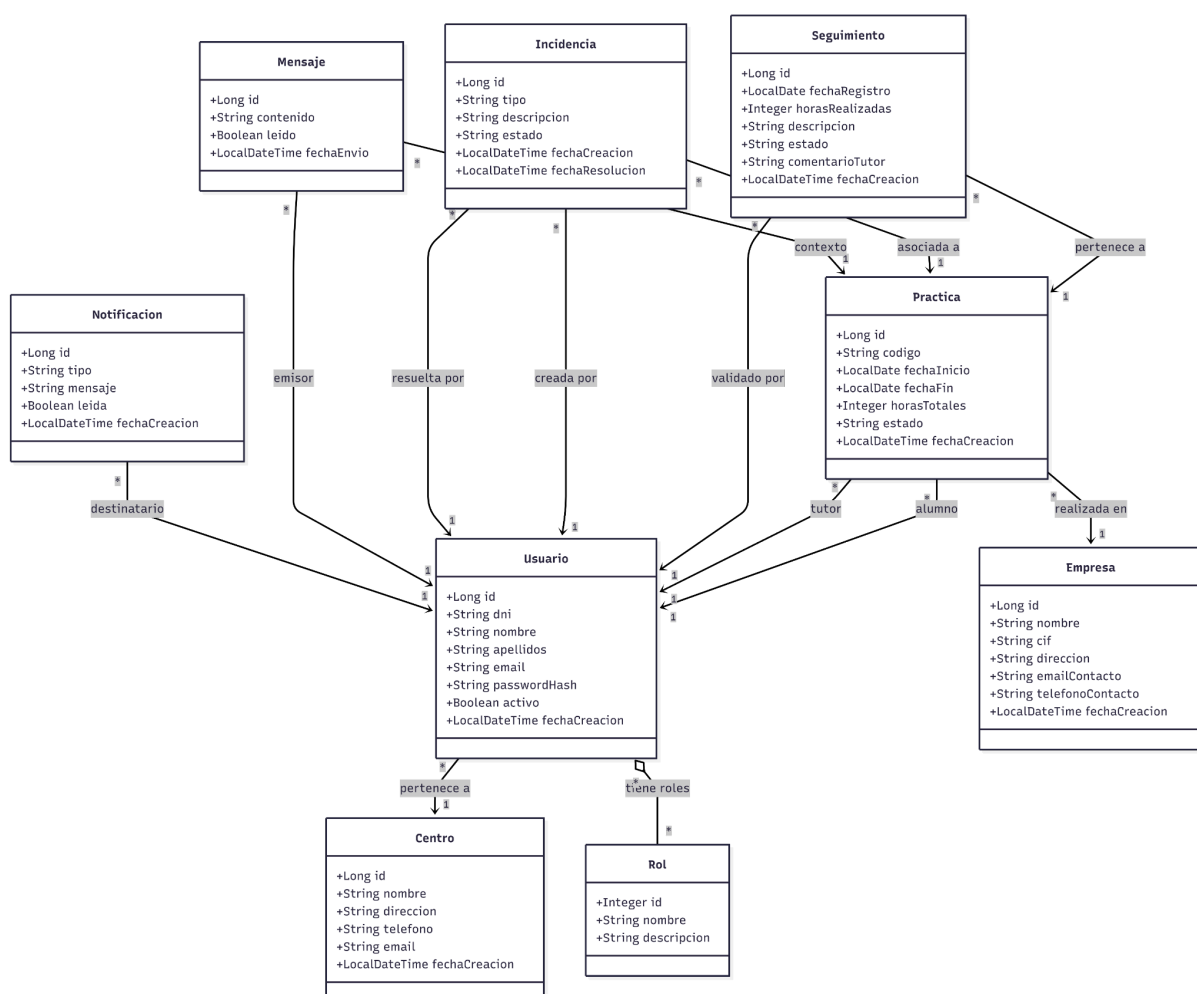
El `docker-compose.yml` configura un healthcheck sobre la base de datos `pg_isready` antes de permitir que el backend arranque. Esto evita errores de conexión durante el inicio en frío, donde PostgreSQL puede tardar unos segundos en estar listo para aceptar conexiones. La variable `JWT_SECRET` está declarada con la sintaxis `${JWT_SECRET:?mensaje}`, de modo que Docker Compose falla de forma explícita si no está definida en el entorno, impidiendo arrancar con un secreto vacío o por defecto.



4.1. Diseño de la Lógica (Backend)

Para organizar el código del servidor, he seguido un patrón orientado a objetos, separando claramente los Controladores (que reciben las peticiones de Flutter), los Servicios (donde está la lógica y las validaciones) y los Repositorios (que hablan con la base de datos).

A continuación, presento el diagrama de clases (UML) del núcleo del sistema, centrado en la autenticación y la gestión de entidades maestras (Centros y Empresas), que constituye el primer gran hito de desarrollo del proyecto.



En este segundo hito le he dado mucha más profundidad al backend. Lo más importante ha sido añadir una seguridad real basada en roles, no solo el login. Ahora cada pantalla de la API tiene una restricción concreta: un alumno con un token válido no puede crear prácticas



ni validar seguimientos aunque lo intente, porque el sistema lo bloquea antes de ejecutar nada. Para conseguir esto he usado la anotación `@PreAuthorize` de Spring Security directamente sobre cada método del controlador.

Para asegurarme de que todo esto funciona correctamente, empecé con una batería de tests de integración que no solo comprueban que las cosas buenas funcionan (que un tutor puede validar un seguimiento), sino también que las cosas malas se bloquean (que un alumno recibe un error 403 si intenta hacer algo que no le toca). Esta batería ha ido creciendo con el proyecto: al cierre del hito 3 cuenta con ciento siete casos cubriendo todos los módulos, con una cobertura medida con JaCoCo del 69,5%. Tener este respaldo me da para seguir añadiendo funcionalidades sin romper lo que ya estaba.

Módulo de ausencias

La implementación del módulo de ausencias surgió hablando con mi tutor en una tutoría del TFG, que me ayudó a detectar que la plataforma registraba el trabajo realizado pero no las faltas de asistencia, que tienen consecuencias distintas sobre el cómputo de horas y la calificación final. Diseñé una tabla independiente en lugar de añadir campos a la tabla de seguimientos existente, porque una ausencia no tiene horas ni descripción de tareas pero sí un fichero adjunto de justificante. Mezclar ambos conceptos habría generado columnas siempre nulas según el tipo de registro, lo que en un modelo relacional indica normalización incorrecta.

El justificante se almacena como `bytea` directamente en PostgreSQL, vinculado al registro de la ausencia. Esta decisión evita la complejidad de gestionar un sistema de ficheros externo para el alcance del proyecto. La respuesta JSON expone solo un campo booleano `tieneJustificante`, de modo que el cliente sabe si existe un fichero sin descargarlo hasta que el usuario lo solicite. El endpoint de descarga verifica antes de devolver los bytes que quien solicita sea participante de la práctica vinculada a esa ausencia.

Panel de administración y sistema de auditoría

El panel de administración permite crear usuarios de todos los roles, editarlos y activarlos o desactivarlos, y también crear y modificar las prácticas activas. La capacidad de edición fue una adición razonada tras analizar los errores más frecuentes durante el despliegue: escribir mal un email o asignar un alumno a la práctica equivocada son errores habituales que no deberían requerir borrar el registro entero para corregirlos. Durante la implementación encontré un error sutil: el servicio usaba `Set.of()` de Java para asignar el nuevo rol al usuario, que devuelve una colección inmutable. Cuando Hibernate intentaba sincronizar la relación `@ManyToMany` de roles llamaba a `clear()` sobre esa colección y lanzaba una



`UnsupportedOperationException`. La corrección fue operar directamente sobre la colección gestionada por Hibernate —`getRoles().clear()` seguido de `getRoles().add(nuevoRol)`— en lugar de sustituir la referencia. Es el tipo de error que parece correcto al leer el código pero solo falla en runtime cuando el framework intenta modificar la colección.

El sistema de auditoría centraliza en la tabla `audit_logs` el registro de todas las operaciones relevantes: usuarios, prácticas, ausencias, incidencias, seguimientos y mensajes de chat. El servicio `AuditService` utiliza `Propagation.REQUIRES_NEW` para que el log de auditoría se guarde en una transacción independiente de la operación principal. Esto garantiza que incluso si la operación falla y su transacción hace rollback, el intento queda registrado —que es exactamente el comportamiento que se necesita en un sistema de trazabilidad.

4.2. El módulo de seguimientos: cuando el diseño inicial no reflejaba la realidad

Aquí quiero explicar algo que me parece interesante porque ilustra bien cómo evoluciona un proyecto real.

Cuando diseñé el módulo de seguimientos al principio, pensé en una validación simple: el alumno registra sus horas, un tutor las aprueba o rechaza, fin. Pero al ponerme a implementarlo en detalle me di cuenta de que ese diseño no reflejaba cómo funcionan las prácticas en la realidad.

El problema es que hay dos tutores con funciones completamente distintas. El tutor de empresa está ahí todos los días con el alumno y sabe exactamente lo que ha hecho esa semana. Él es quien firma el parte en papel actualmente. El tutor del centro, en cambio, no está en la empresa: su trabajo es supervisar que el proceso formativo va bien, que el alumno no tiene problemas, y que se están cumpliendo los objetivos del ciclo. Son cosas distintas y mezclarlas en una sola validación no tenía sentido.

La solución que adopté fue un flujo en dos pasos. Primero valida el tutor de empresa (¿son correctas las horas y las tareas?). Solo después de eso puede revisar el tutor del centro (¿va bien el alumno en general?). Para que esto funcione, el parte puede estar en cuatro estados distintos: esperando la firma de la empresa, esperando el visto bueno del centro, completado, o rechazado.

El caso del rechazo también lo pensé con cuidado. Si la empresa rechaza un parte, el sistema crea automáticamente una incidencia que el tutor del centro puede ver sin que el



alumno tenga que hacer nada. Esto es importante porque un rechazo puede ser una tontería (el alumno puso mal las horas), pero también puede indicar que algo va mal con la empresa. En cualquier caso, el tutor del centro tiene que saberlo.

Detectar este problema antes de construir las pantallas fue una suerte, porque cambiarlo ahora tiene un coste bajo. Si lo hubiera descubierto después de tener el frontend hecho, habría tenido que rehacer todo.

4.3. Seguridad aplicada: revisión sistemática OWASP

Durante el tercer hito realicé una revisión de seguridad estructurada siguiendo el estándar OWASP Top 10 (2021), aplicando las correcciones directamente al código en lugar de dejarlas para una fase final. Recojo a continuación las decisiones más relevantes.

Control de acceso (A01). Varios controladores REST incluían `@CrossOrigin(origins = "*")`, que permite peticiones desde cualquier origen y anula la protección CORS frente a peticiones maliciosas. Eliminé estas anotaciones y centralicé la configuración en `SecurityConfig`, especificando solo los orígenes legítimos. También corregí dos expresiones de `@PreAuthorize` que referenciaban una propiedad inexistente del objeto `UserDetails`; las sustituí por llamadas al servicio que verifican si el usuario autenticado es participante de la práctica solicitada.

Fallos criptográficos (A02). El método de firma JWT usaba `secret.getBytes()` para obtener la clave. Como el secreto está en Base64, `getBytes()` trata los caracteres de esa codificación como bytes literales, no los bytes reales que representan. La corrección fue `Decoders.BASE64.decode(secret)`. Los tokens generados con el método antiguo son incompatibles con los del nuevo, lo que obligó a invalidar las sesiones durante el despliegue del fix. Además reforcé la política de contraseñas con `@Pattern` en el DTO de registro: mayúscula, minúscula, dígito, carácter especial y mínimo diez caracteres.

Diseño inseguro (A04). El servicio de seguimientos no impedía registrar varios partes en la misma semana ISO, lo que generaría inconsistencias en el cómputo de horas. Añadí la comprobación sobre la semana (lunes a domingo) antes de permitir un parte nuevo. El módulo de ausencias aplica un control equivalente: no puede existir más de una ausencia para la misma práctica y fecha.

Fallos de autenticación (A07). El endpoint de registro devolvía mensajes de error distintos según si el campo duplicado era el email o el DNI, lo que permite enumerar qué datos existen en el sistema. La corrección fue unificar la comprobación y devolver siempre el mismo mensaje genérico. Para el cierre de sesión implementé una blacklist de tokens en



servidor: cada JWT incluye un claim `jti` con un UUID único, y al hacer logout ese identificador se registra en memoria. El filtro de autenticación verifica la blacklist antes de aceptar cualquier token, invalidando inmediatamente un token robado aunque no haya caducado.

Rate limiting y cabeceras (A05). Un filtro con máxima prioridad en la cadena de Spring Security limita a diez las peticiones de autenticación por IP y minuto, devolviendo HTTP 429 al superarlo. Las cabeceras `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` y una `Content-Security-Policy` restrictiva se configuraron tanto en Spring Security como en Nginx, cubriendo tanto las respuestas de la API como la carga de la aplicación web.

Control de acceso a nivel de objeto (A01 — IDOR). Durante la revisión final del proyecto detecté que los servicios de seguimiento y ausencias verificaban el rol del usuario —TUTOR_EMPRESA o TUTOR_CENTRO— pero no comprobaban que el usuario autenticado fuera el tutor asignado específicamente a la práctica sobre la que estaba actuando. Cualquier tutor de empresa con sesión activa podía validar partes de prácticas que no le correspondían. La corrección añade una verificación explícita antes de cualquier modificación de estado:

```
practica.getTutorEmpresa().getEmail().equals(emailAutenticado).
```

 Si la comprobación falla, el servicio lanza `AccessDeniedException` antes de ejecutar nada. Es la diferencia entre "¿tienes el rol correcto?" —control por tipo de usuario— y "¿tienes permiso sobre este recurso concreto?" —control por pertenencia—, que OWASP A01 denomina Broken Object Level Authorization. El mismo patrón se aplica al listado de seguimientos y ausencias por práctica: aunque el endpoint sea de solo lectura, un usuario sin relación con esa práctica recibe un 403 en lugar de los datos.

4.4. Tests automatizados y cobertura de código

Desde el inicio del proyecto traté los tests de integración como parte del desarrollo, no como un añadido opcional. La arquitectura en capas facilita esto: los servicios encapsulan toda la lógica de negocio de forma independiente al protocolo HTTP, lo que permite testearlos directamente con un contexto Spring completo sobre una base de datos H2 en memoria sin necesidad de arrancar el servidor. Al finalizar el tercer hito el proyecto cuenta con ciento veintiún tests organizados en doce clases. Los tests de servicio usan `@SpringBootTest` con el perfil `test`, que activa H2 con compatibilidad PostgreSQL y desactiva Flyway para que Hibernate genere el esquema. Los tests de controlador usan `@WebMvcTest`, que levanta solo la capa web y permite verificar el comportamiento de seguridad por roles con `@WithMockUser`, comprobando que un endpoint devuelve 403 cuando lo llama un rol sin permiso sin necesidad de hacer una petición real.



Para medir la cobertura configuré JaCoCo, que instrumenta el bytecode en tiempo de compilación. Ejecutar `./mvnw verify` corre todos los tests y genera el informe en `target/site/jacoco/index.html`. La cobertura al cierre del Hito 3 es del 69,5% de instrucciones, con los módulos principales —ausencias, incidencias, administración— por encima del 80%.

Durante la escritura de los tests detecté un error que había pasado desapercibido: la implementación de seguimientos accedía a `SecurityContextHolder.getContext().getAuthentication().getName()` sin verificar si la autenticación era nula. En los tests de integración ese método se llama sin contexto de seguridad activo y lanza un `NullPointerException`. La corrección fue un método privado `currentUserEmail()` que devuelve `"system"` cuando no hay autenticación. Es un ejemplo de cómo la escritura de tests no solo verifica funcionalidad, sino que obliga a revisar el código desde un ángulo diferente y aflora bugs que la ejecución normal no ejercita.

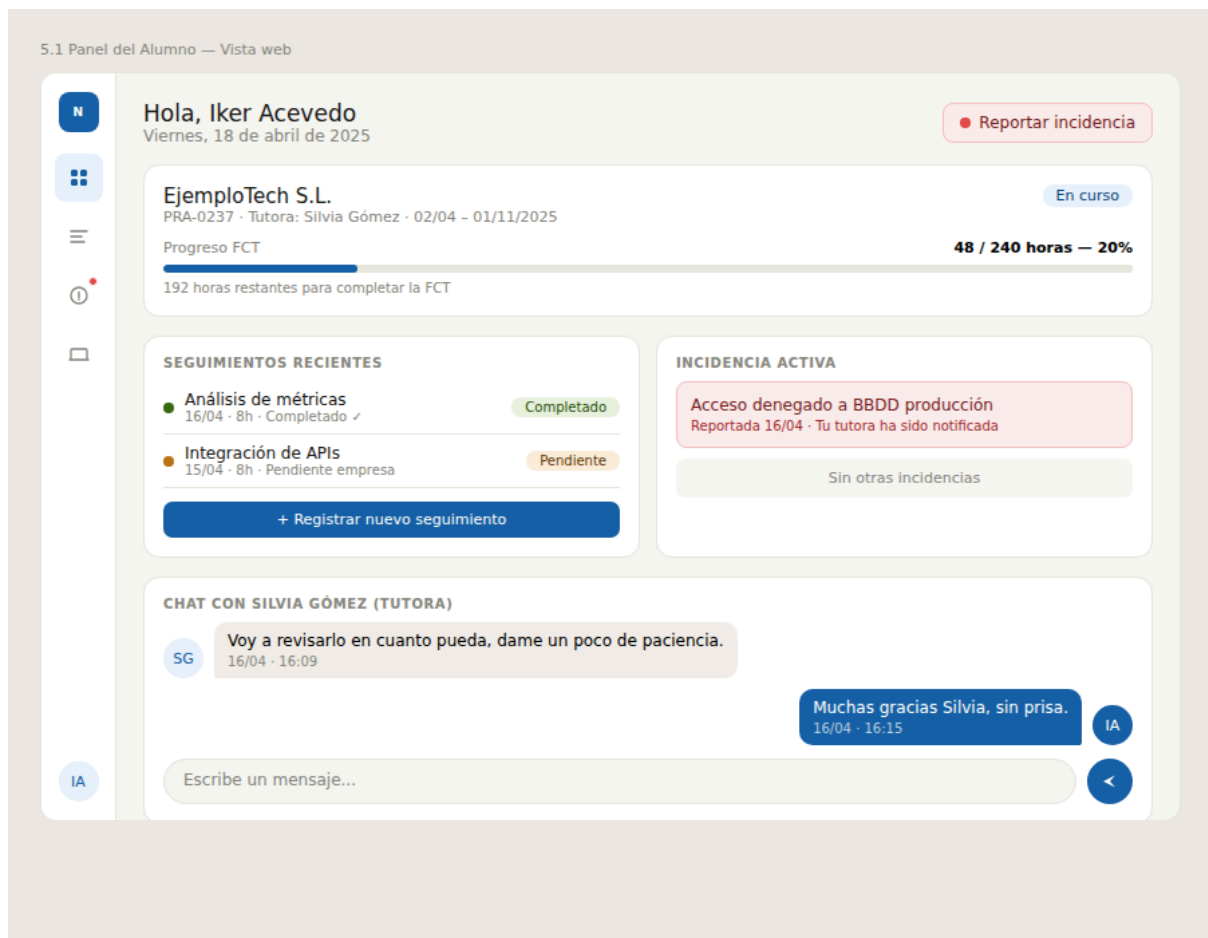
Al cierre del proyecto, la batería cuenta con **ciento veintiún tests** organizados en catorce clases. Los catorce nuevos casos incorporados en el hito final cubren el módulo de chat: `MensajeServiceTest` verifica el almacenamiento y recuperación de mensajes con su práctica y remitente asociados, y `MensajeControllerTest` comprueba el comportamiento de seguridad por roles —que un alumno solo puede enviar mensajes a prácticas en las que participa, y que un usuario sin autenticación recibe un 401 antes de llegar al controlador—. La cobertura global se mantiene en el 69,5% de instrucciones medido con JaCoCo.



5. Diseño de la Interfaz por Roles

Aquí es donde se ve cómo funciona la aplicación. He diseñado cuatro paneles distintos porque cada usuario necesita cosas diferentes. Y luego he generado también la versión móvil.

5.1. Panel del Alumno



- **Contador de Horas:** Una barra de progreso que te dice de un vistazo cuánto te queda para terminar.
- **Seguimiento Semanal:** Un espacio para escribir qué has hecho cada día. Esto elimina los informes pesados al final del mes.
- **Botón de Incidencias:** Si hay un marrón (tareas que no tocan, problemas de horario), se reporta aquí oficialmente.
- **Chat:** Hablar con el tutor de forma directa y profesional.



5.2. Panel del Tutor

5.2 Panel del Tutor del Centro — Vista web

Mis alumnos
3 prácticas activas

- JP José Pérez
EjemploTech S.L. 11
- LS Laura Sánchez
Innovatech Solutions Rev.
- CR Carlos Ruiz
SoftDevia S.A. OK

SG

José Pérez
EjemploTech S.L. · PRA-0237 En curso

Progreso FCT 48 / 240 h — 20%

SEGUIMIENTO PENDIENTE DE VALIDAR (2ª FASE)

- Integración de APIs externas
15/04 · 8h · Ya validado por empresa ✓ Rechazar Dar visto bueno

INCIDENCIA ABIERTA
Acceso denegado a BBDD producción
Reportada 16/04 · Sin resolver · Pendiente intervención
Gestionar incidencia

CHAT CON JOSÉ PÉREZ

JP Tengo problemas con las credenciales de BBDD, me da error.

Voy a revisarlo en cuanto pueda. SG

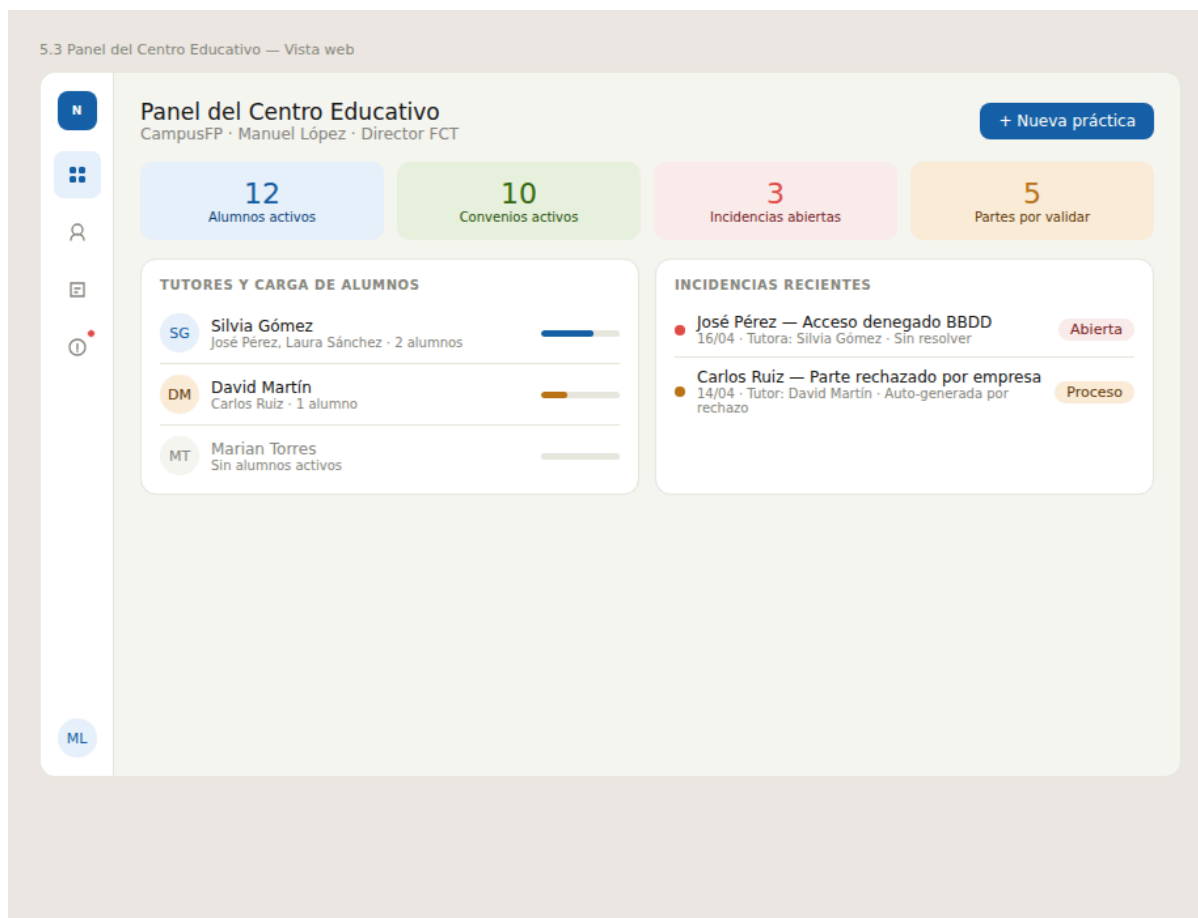
Escribe un mensaje...

- **Lista de Alumnos:** El tutor puede saltar de un alumno a otro para ver su progreso FCT, los partes pendientes de su validación final y las incidencias abiertas.
- **Alertas:** Indicadores visuales (como puntos rojos) que avisan si alguien ha reportado una incidencia o no ha rellenado el diario.
- **Lista de Alumnos:** El tutor puede saltar de un alumno a otro para ver su progreso FCT, los partes pendientes de su validación final y las incidencias abiertas.
- **Partes pendientes:** Vista global de todos los seguimientos que están esperando la validación final del centro, sin necesidad de entrar en cada alumno.
- **Alertas visuales:** Cada alumno en la lista lleva un badge numérico rojo que suma sus incidencias abiertas y ausencias injustificadas, para detectar casos que necesitan atención sin revisar ficha por ficha.
- **Barra de progreso FCT:** Muestra las horas completadas frente al total del convenio,



computando únicamente seguimientos en estado COMPLETADO —los que ya tienen las dos validaciones—, evitando inflar el progreso con partes aún pendientes.

5.3. Panel del Centro Educativo / Admin



- **Gestión de Tutores:** Ver cuánta carga de trabajo tiene cada profesor.
- **Estadísticas:** Un resumen de cuántos alumnos hay en prácticas y qué convenios están activos.
- Gestión de usuarios: Crear usuarios de cualquier rol, editarlos (nombre, email, DNI, rol) y activarlos o desactivarlos sin necesidad de eliminar el registro.
- Gestión de prácticas: Crear y modificar prácticas activas, corrigiendo participantes o fechas sin borrar y recrear el convenio.
- Auditoría: Historial de todas las operaciones del sistema, filtrable por módulo, con el email del actor y la descripción de cada acción. Accesible solo por el administrador.



5.4. Panel del Tutor de Empresa

Al añadir la doble validación, necesité diseñar una pantalla específica para el tutor de empresa. He decidido mantenerla deliberadamente simple porque el tutor de empresa no necesita ver el historial académico del alumno, ni las incidencias del centro, ni el detalle del chat. Si le meto todo eso en una pantalla se vuelve confusa y deja de parecerse al proceso que ya conoce: revisar el parte de la semana y firmarlo.

El panel se organiza en dos pestañas accesibles desde la barra lateral.

La primera pestaña, **Partes pendientes**, muestra tres indicadores en la cabecera: el número de partes que esperan firma, las horas que el tutor ya ha validado en el convenio activo, y las horas que le quedan al alumno hasta completarlo. Debajo aparecen los partes ordenados por fecha, con la descripción que escribió el alumno y los botones de validar o rechazar. Cuando rechaza un parte tiene que escribir el motivo obligatoriamente, de modo que queda registrado y el tutor del centro sabe exactamente qué ocurrió. Aparte, también le saltan los faltas que registre el alumno, y podrá ver el justificante adjunto a la falta en caso de que el alumno lo haya adjuntado.

La segunda pestaña, **Progreso del alumno**, está pensada para los momentos en que el tutor quiere tener una visión global sin entrar en el detalle de cada parte. Muestra una tarjeta por práctica con el nombre del alumno, el código del convenio, las fechas de inicio y fin, y una barra de progreso visual que representa las horas validadas frente al total comprometido. Bajo la barra aparece el desglose numérico: horas registradas por el alumno, horas ya validadas por la empresa, y horas que aún quedan para completar el convenio. Los indicadores de la primera pestaña se calculan únicamente sobre prácticas en estado activo, evitando que convenios ya finalizados distorsionen los totales.



5.4 Panel del Tutor de Empresa — Vista web

N



Partes pendientes de firma

Marco Rodríguez · EjemploTech S.L.

2

Pendientes de firma

8

Firmados este mes

48h

Horas acumuladas

PARTES DE SEGUIMIENTO — JOSÉ PÉREZ

JP

Semana del 14 al 18 de abril

8 horas · Integración de APIs externas con autenticación OAuth

Pendiente firma

"He trabajado en la integración con tres APIs externas del cliente. Configuré los interceptores de Dio para gestionar la autenticación OAuth de forma automática en todas las peticiones."

Rechazar (indicar motivo)

✓ Validar y firmar

JP

Semana del 7 al 11 de abril

8 horas · Análisis y diseño del módulo de métricas

Pendiente firma

Rechazar (indicar motivo)

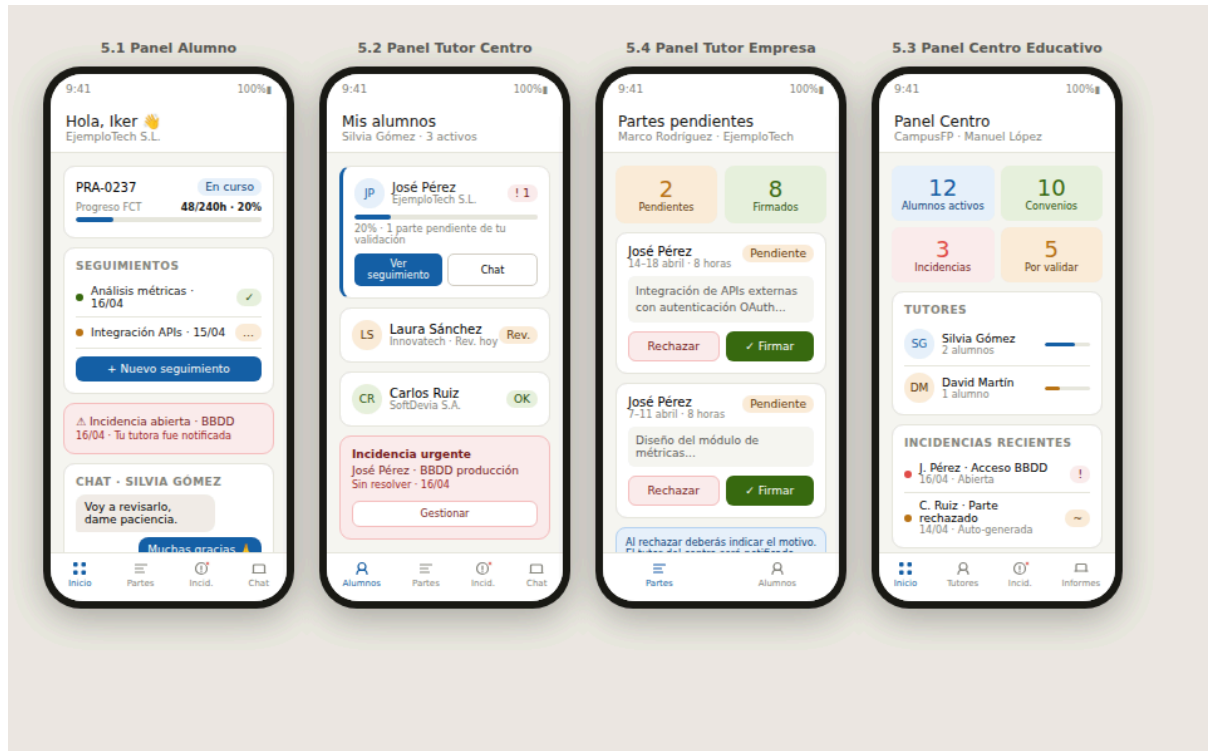
✓ Validar y firmar

¡ Al rechazar un parte deberás indicar el motivo. El tutor del centro será notificado automáticamente sin que el alumno tenga que hacer nada.

MR



5.5. Los cuatro paneles en versión móvil



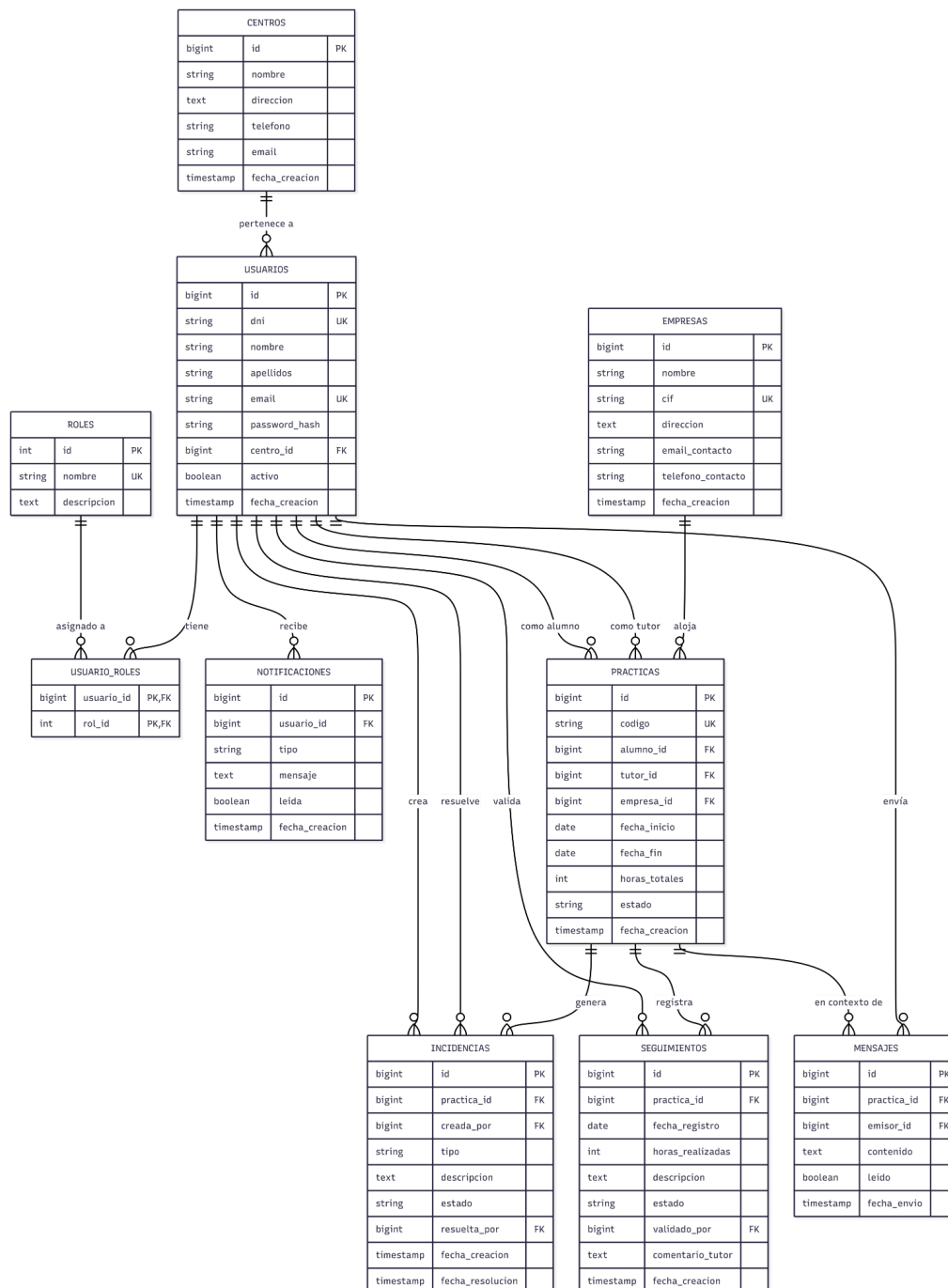
6. Modelo de la Base de Datos

Para que todo esto funcione por detrás, he diseñado un sistema de tablas conectadas:

- **Usuarios:** Donde guardamos quién es alumno, quién es tutor y quién es del centro.
- **Prácticas:** La tabla principal que une al alumno con su empresa y su tutor.
- **Seguimientos e Incidencias:** Registros de lo que se va haciendo y de los problemas que surgen.
- **Chat:** Guardamos todos los mensajes para que haya un historial de lo hablado.

Durante el desarrollo, detecté que un alumno de FP necesita dos responsables: el **Tutor del Centro** y el **Tutor de la Empresa**. He actualizado el modelo para separar ambas figuras en la tabla de prácticas, garantizando un seguimiento académico y laboral independiente..

El resultado final de las tablas interconectadas en PostgreSQL es el siguiente diagrama de Entidad-Relación:



Varias decisiones de diseño del esquema merecen una explicación más detallada.

El justificante de ausencia se almacena como `bytea` directamente en PostgreSQL, vinculado al registro de la ausencia. Esta decisión evita la complejidad de gestionar un sistema de ficheros externo —con rutas, permisos y sincronización— para el alcance del proyecto. La respuesta JSON expone únicamente un campo booleano `tieneJustificante`, de modo que el cliente sabe si existe un fichero sin descargarlo



hasta que el usuario lo solicita explícitamente. El endpoint de descarga verifica antes de devolver los bytes que quien hace la petición sea participante de la práctica vinculada, aplicando el mismo control de acceso a nivel de objeto que en el resto de módulos.

La tabla `audit_logs` almacena todas las operaciones relevantes del sistema con cuatro campos clave: el módulo afectado, la acción ejecutada, el identificador del recurso y el email del actor. El servicio de auditoría utiliza `Propagation.REQUIRES_NEW` para escribir en una transacción independiente de la operación principal. Si la operación falla y hace rollback, el intento queda igualmente registrado en el log —que es exactamente el comportamiento necesario en un sistema de trazabilidad.

La tabla de mensajes del chat vincula cada mensaje a una práctica concreta y a su remitente. Al cargar la pantalla del chat se recupera el historial completo mediante REST; los mensajes posteriores se reciben en tiempo real por suscripción al topic `/topic/practica/{id}` de STOMP. Solo los participantes de esa práctica están suscritos a ese topic, garantizando que los mensajes no se cruzan entre prácticas distintas.

En esta fase se han completado las entidades JPA en el backend, asegurando una sincronización total entre el código y el diagrama Entidad-Relación. Esto incluye la implementación de las clases Java para Incidencias, Mensajes y Notificaciones, que estaban definidas en la base de datos pero pendientes de desarrollar en el servidor.

7. Planificación (Cómo lo voy a hacer)

He dividido el trabajo en 16 semanas para llegar bien a la entrega:

1. **Semanas 1-3:** Analizar todo bien y diseñar las tablas de la base de datos.
2. **Semanas 4-7:** Programar el cerebro (backend) y el sistema para entrar en la web (login).
3. **Semanas 8-12:** Montar las pantallas (frontend), el chat y el sistema de incidencias.
4. **Semanas 13-16:** Probar que todo funcione, corregir fallos y terminar de escribir esta memoria.

Estado Actual (Entrega Hito 1 - 25%): En la primera entrega se completaron las Semanas 1 a 3, con el análisis y diseño de la base de datos mediante Flyway en PostgreSQL, y se avanzó drásticamente en las Semanas 4 a 7. El backend contaba con la estructura base, el sistema de cifrado de contraseñas, la generación de tokens JWT y los endpoints funcionales para el login, el registro y la consulta de entidades maestras.



Estado Actual (Entrega Hito 2 - 50%): Se ha alcanzado el ecuador del proyecto. El backend cuenta ahora con la lógica core completa: CRUD de prácticas con control de estados, sistema de seguimientos con validación por tutores, seguridad real basada en roles con `@PreAuthorize` en cada endpoint, y una batería de 10 tests de integración que verifican los flujos críticos del sistema. En el frontend Flutter se ha superado la fase de esqueleto inicial: la pantalla de login está conectada a la API en tiempo real, el dashboard muestra los datos reales de la práctica activa del alumno, y el cliente HTTP está configurado con interceptores que inyectan el token JWT automáticamente en cada petición. Durante esta fase se detectó y corrigió un error de diseño en el flujo de validación de seguimientos, separando las responsabilidades del tutor de empresa y el tutor del centro en dos fases diferenciadas, lo que refleja con mayor fidelidad el proceso real de las FCT.

El rediseño del módulo de seguimientos no estaba planificado inicialmente, pero detectarlo a tiempo fue clave. Cambiarlo ahora, antes de tener las pantallas construidas, fue relativamente sencillo. Habría sido mucho más costoso descubrirlo al final.

Estado Actual (Entrega Hito 3 — 75%): El tercer hito concentró el mayor volumen de trabajo del proyecto. Se completaron cuatro bloques funcionales independientes: el módulo de ausencias con su flujo de revisión y gestión de justificantes adjuntos, el panel de administración con edición completa de usuarios y prácticas, el sistema de auditoría centralizado, y una revisión sistemática de seguridad OWASP que resultó en correcciones concretas sobre CORS, firma JWT, política de contraseñas, gestión de sesiones y rate limiting. La batería de tests creció hasta ciento siete casos con cobertura del 69,5% medida con JaCoCo. Dos aspectos merecen mención especial: el error

`UnsupportedOperationException` causado por `Set.of()` inmutable en la edición de usuarios —un bug que solo se manifiesta en runtime cuando Hibernate intenta modificar la colección— y la transacción independiente del servicio de auditoría con `Propagation.REQUIRES_NEW`, que garantiza que los intentos fallidos también quedan registrados.

Estado Actual (Hito 4 — en desarrollo): El cuarto hito cierra el ciclo de comunicación que era el objetivo central del proyecto desde su definición inicial. Se implementó el chat en tiempo real mediante WebSocket con el protocolo STOMP: el backend expone el endpoint `/ws`, la autenticación se resuelve pasando el token JWT en las cabeceras del frame STOMP CONNECT, y un interceptor de canal lo valida antes de registrar la conexión. Los mensajes se publican en topics por práctica —`/topic/practica/{id}`— de modo que solo los participantes de cada práctica reciben los mensajes de su canal. El historial se carga al abrir la pantalla mediante REST y los mensajes nuevos llegan por WebSocket sin necesidad de polling.

Junto al chat, este hito incorporó varias mejoras de seguridad y calidad detectadas durante la revisión final. Se corrigió una vulnerabilidad IDOR en los servicios de seguimiento y ausencias: el sistema verificaba el rol del usuario pero no que fuera el participante asignado a esa práctica concreta. Se implementó la restauración de sesión en Flutter —`GET /auth/me` al arrancar la app— para que el usuario no tenga que autenticarse de nuevo si su token sigue siendo válido. La blacklist de tokens y el mapa de rate limiting recibieron una



limpieza periódica automática mediante `@Scheduled`, evitando el crecimiento indefinido de esas estructuras en memoria. La batería de tests creció hasta ciento veintiún casos con la incorporación de los tests del módulo de chat.

8. Conclusión y Futuro

Este proyecto demuestra que se puede mejorar significativamente la experiencia de las prácticas si se centraliza en un único entorno digital lo que antes estaba disperso entre correos, llamadas y hojas de cálculo. La plataforma Nexus cubre el ciclo completo del seguimiento: el registro semanal de actividades con validación en dos pasos por figuras con responsabilidades distintas, la gestión de incidencias y ausencias con justificante adjunto, la comunicación directa en tiempo real entre los participantes de cada práctica, y un sistema de auditoría que registra todo lo que ocurre en el sistema.

Uno de los aprendizajes más valiosos ha sido comprobar que las decisiones con más impacto no son las tecnológicas sino las de lógica de negocio. Detectar que el flujo de validación necesitaba dos pasos diferenciados antes de construir ninguna pantalla, o que una ausencia y un seguimiento son entidades con naturaleza distinta que no deben compartir tabla, son decisiones que no se ven en la interfaz pero que hacen el sistema correcto y mantenible. La revisión de seguridad OWASP fue igualmente reveladora: algunos problemas como el wildcard CORS o el uso incorrecto de `getBytes()` para la clave JWT son errores que se cometen con naturalidad al seguir tutoriales y solo se detectan cuando se entiende el porqué detrás de cada control.

En cuanto al futuro, la plataforma está pensada para crecer en dos direcciones: a corto plazo, firma digital de convenios y notificaciones push; a medio plazo, la gestión del proceso previo a las prácticas —publicación de perfiles por la empresa, preferencias del alumno y asignación por el centro— que convertiría Nexus en la plataforma completa del ciclo de prácticas de principio a fin.